

Dynamic constraint satisfaction for feature interaction

C D Elfe, E C Freuder and D Lesaint

As telecommunications service features become more plentiful, service providers are faced with the increasingly difficult task of providing consistent and high-quality service. A stumbling block along the path towards making available large numbers of complex and potentially highly customised features are the unwanted or unforeseen behaviour of these features when operating together. This problem, known as the feature interaction problem, is only compounded by the heterogeneity of service providers, service platforms, and feature implementations.

Presented in this paper is a constraint-based approach for performing avoidance, detection, and resolution of these feature interactions in a run-time telecommunications environment, such as an intelligent network. This constraint representation centres around a framework for call resources, permitting structured access to each of the aspects of call functionality, allowing features to remain modular. The representation is further enhanced, beyond the basic finite state feature processes, by the maintenance of context information. The resulting call session information keeps track of information such as how a feature is being used, by whom it was invoked and when, and what other callers and features are currently involved.

1. Introduction

Telecommunications systems, no longer embedded within highly specialised hardware, exist today as reconfigurable and flexible computer networks. Since new functionality can be realised in software, it would appear that large numbers of customised features should be available. However, one of the prices for this freedom is the feature interaction problem. The potential problem is that different features, or multiple instances of the same feature, can cause unforeseen and unintended behaviour when operating together.

Efforts can certainly be taken during the design of a new feature, but since different implementations of even the same feature can function in slightly different ways, testing can become quite difficult. In fact, even with a few callers and a small number of features, the number of possible scenarios can make exhaustive testing impractical. Yet, telecommunications users expect the same quality of service to exist whether their connection is local or global, regardless of the number of service providers that might be involved.

The inability to be able to predict or remove all interaction during feature development underscores the need for a mechanism to deal with run-time feature interaction.

1.1 Feature interaction example

Consider the two common features provided to phone customers — call waiting (CW) and call forwarding on busy (CFB). Call waiting notifies the call recipient, who is already on the line, of an incoming call and allows the recipient to switch between the two calls, connecting with one, suspending the other. Call forwarding on busy engages when the recipient is already on the line and automatically directs calls to another number. Both CW and CFB allow incoming calls to connect normally when the recipient is not busy.

Separately, each feature can operate without any interaction. However, when both features are activated and the recipient is busy, interaction occurs, due to the fact that neither feature can be successfully executed without sabotaging the operation of the other. Specifically, there are two different and conflicting actions to be performed when the call recipient is busy as shown in Fig 1.

1.2 Dealing with interaction

In the days when telephone telecommunications consisted almost exclusively of basic telephony, well thought-out, detailed finite state diagrams, such as those in ITU standards document Q.1204 [1], were sufficient to

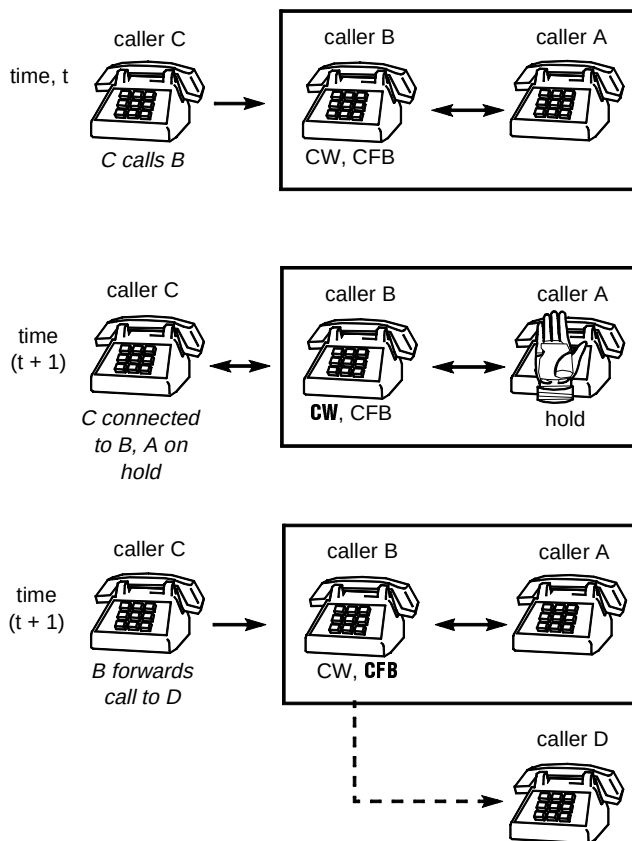


Fig 1 Interaction arises at time $(t + 1)$: is the call from C answered by B, putting A on hold, or is C forwarded to D?

describe its behaviour and guarantee proper function. In spite of the fact that known feature interactions continue to be enumerated, discussed, and categorised, as in Cameron and Velthuijsen [2], Wakahara et al [3], and Cameron et al [4], comprehensive solutions and formalisms are still elusive.

Telephony and features are quite naturally described by their finite state behaviour. Attempts to capture feature interaction within finite state models have resulted in several variations, such as Au and Atlee [5], Iraqi and Erradi [6], Khoumsi [7], and Klein et al [8], where interactions are detected by performing reachability analysis, or checking for deadlock, absorbing states, or non-determinism. Since exhaustive testing can be unfeasible, due to the state explosion problem, good approximation methods are also being pursued, such as with the use of Petri-nets [9].

Formal approaches, like Thistle et al [10] and Frappier et al [11], have attempted to perform inferences on the effects feature I/O have on calls and on each other, but still encounter the difficulty of defining exactly what is involved in the composition of two or more features, as in Mierop et al [12] and Bergstra and Bouma [13].

Simulated execution of features has also been attempted, as in Boumezbeur and Logrippo [14] and Capellmann et al [15]. Other modelling, where the focus is closer to implementations, can be seen in the verification of

learned run-time behaviour of Tsang and Magill [16] and in the run-time negotiation between agents of Griffeth and Velthuijsen [17].

1.3 Constraint-based avoidance, detection, and resolution

In spite of the numerous, clever approaches to deal with the feature interaction problem, the need for a comprehensive medium remains, not only within which to represent telecommunications, but also to be able to avoid, detect, and resolve feature interaction. Constraint satisfaction technology could possibly be such a unifying element, since our initial efforts have led to success in each of the three basic areas. Constraints have been used to model interaction of the following types.

- Feature ordering

Feature behaviour is triggered by certain call conditions, yet features by design modify call functionality. When multiple features are active at one time, there is the potential for one feature to unfavourably alter the state of the call for another feature, when processed in one particular order, yet operate without conflict in another. In cases where interaction occurs due to an ordering issue, both the constraint representation and search mechanism employed offer the ability to prevent interaction when possible and detect interaction where unavoidable.

- Resource contention

Interactions can also occur due to forms of resource contention, like the well-known one in Fig 1. By employing a constraint representation to model the call resources in a uniform manner, within a call framework, as suggested by Utas [18], interactions involving resources can be detected. In some instances, due to the type of conflict, the interaction cannot be avoided, yet resolution involving an alternative or diminished set of features can be performed. An advantage of a constraint representation in this case is that the same mechanism for detecting the interaction can be employed to perform the resolution.

- Context violation

Calls involving multiple callers and features are susceptible to certain types of interaction, particularly those resulting from ambiguous signal interpretation or mistaken roles of callers. The reason for these types of interaction are the various and different contexts created by the activation of each new feature and by the inclusion of each new caller into the call. By the application of context-related constraints to both feature and caller information, the relationships between callers and features become clearer and help expose the interaction. In some instances, the context information required to detect an interaction might

dispel the ambiguity and result in a trivial resolution. In those cases when resolution is not so obvious, the context information provides an excellent starting point for informed, intelligent resolution.

1.4 Constraint satisfaction

Constraint satisfaction emerged out of early work in computer vision. Since that time, many algorithms have been developed to perform search and inferencing on problems represented as constraint satisfaction problems (CSPs). Constraint technology has been successfully applied to various areas, such as planning, scheduling, resource allocation, and diagnosis, an area that perhaps most closely resembles feature interaction.

As constraint satisfaction has already been used to verify network protocols [19, 20], it is not surprising that feature interaction should be added to the list. By equating consistency of a constraint representation with an interaction-free state of the features in the telecommunications system, a constraint-based mechanism can be used to verify the proper functioning of the telecommunications system with each signal.

1.5 Classical CSPs

A constraint satisfaction problem consists of variables, values, and constraints. Each variable has a finite set of domain values, and the constraints between variables represent permitted assignment tuples. In Fig 2, the variables are ringer, handset, and connect status. The rounded boxes give their domain values, and the connecting lines are the constraints, which represent the permitted combinations of values.

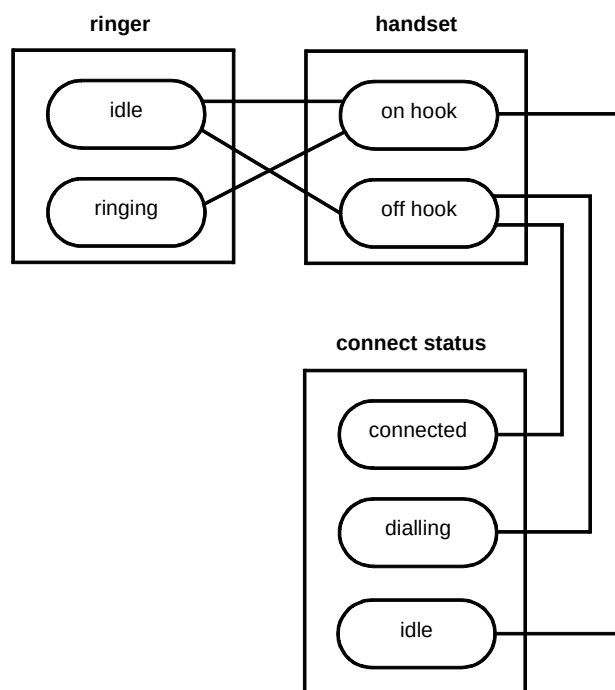


Fig 2 A classical constraint satisfaction problem.

A solution to a CSP is a labelling of the variables, in such a way that the constraints are satisfied. For example, a solution to the CSP in Fig 2 is the labelling *idle*, *on hook*, *ringing* for the variables 'connect status', 'handset', 'ringer', respectively. However, *dialling*, *off hook*, *ringing* is not a solution, because, when the variable 'handset' has the value *off hook*, there is no constraint permitting 'ringer' to take on the value *ringing*.

1.6 Dynamic CSPs

During search, the labels for variables are assigned and then potentially reassigned as new possibilities are pursued. In a classical CSP, the problem's variables, values, and constraints are fixed at the outset and do not change. In an environment like telecommunications, however, the assumption of a static problem is not valid. The arrival of new signals, the addition of new callers, activation and deactivation of features from users' profiles create a highly dynamic environment. To reflect changes that have occurred requires an extension of classical CSP, known as dynamic CSP. A number of standard algorithms for classical CSPs have been extended to deal with dynamic CSPs.

The problem in Fig 3 is an example of a dynamic CSP. Due to the state of the system, the value of the variable 'incoming signal' changed. This event has invalidated the existing solution (shown as shaded values), requiring that a new solution be computed.

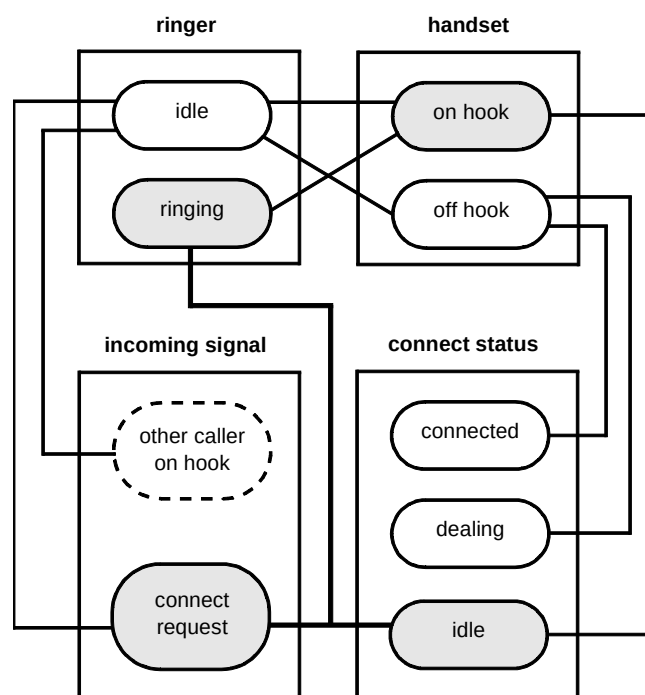


Fig 3 A dynamic constraint satisfaction problem.

2. Feature ordering

In basic telephony, signal handling is very clearly specified. However, signal handling in the presence of features is not. Features are triggered by certain call states and configurations, yet by definition modify a call's behaviour. In the presence of multiple active features, the state of a call can change a great deal, and the order in which the features are processed can lead to vastly different results, some possibly unwanted.

2.1 Example

Consider the interaction that can occur, due to ordering, when the two features 'originating call screening' and 'speed dialling' are present. Originating call screening (OCS) allows a caller to prevent outgoing calls to be placed to certain numbers. Speed dialling (SD) allows for shorthand key-strokes to be saved as encodings for longer numbers, allowing the user to minimise the number of keys pressed.

If OCS is applied before SD, then it is the untranslated, short-hand number that is checked against the denied list of numbers. After this, the shorthand number is translated by SD, and the call is placed to the denied number. This undesirable behaviour is shown in Fig 4.

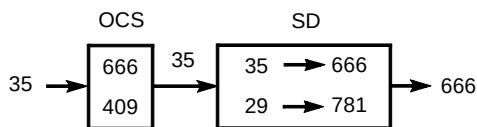


Fig 4 OCS fails to prevent the connection to a denied number, due to the order in which features are processed. This is because OCS checks the untranslated number 35 dialled first, and subsequently SD translates it into the denied number 666 which is used to make the connection.

Clearly, the intended behaviour is for translation to have taken place first and then for the number to be verified. Specifically, the ordering should have had SD first, followed by OCS.

2.2 Avoidance and detection

In the proposed system, all features are represented within a call framework, which contains call resources. Additionally, each role that a caller plays with respect to other callers and features is maintained explicitly by separate call session information. Both of these representations not only allow for telephony and features to be more accurately modelled, but also provide the ability to expose interaction.

For example, consider OCS and SD. These two features have the potential to cause an interaction, even when they should not, in the case that a valid number is dialled. However, the constraint-based mechanism can prevent unwanted behaviour and enforce the proper order. In fact, the features can be developed independently within the framework, without their interaction being known ahead of

time. By checking the constraints, the constraint mechanism discovers that OCS-SD ordering is invalid and that SD-OCS is the correct one.

Consider the example in Fig 5 where the valid number 29 is given. The signal Digits_Dialled_Sent has arrived and is being processed by the process handler for that user. The small numbers indicate the order in which the signal is being processed. OCS will process the dialled input number 29 first, which is residing in the Terminating_Number variable of session 1. After this, OCS will immediately block any other feature from modifying the value of the Terminating_Number variable.

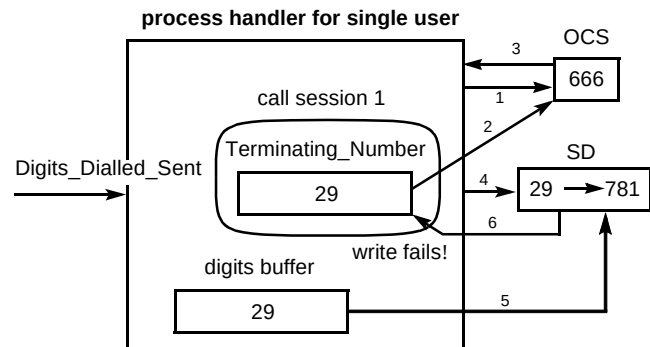


Fig 5 The signal, having been allowed to propagate according to the specified order, has resulted in an inconsistency. This was accomplished because the constraints placed on the Terminating_Number by OCS caused the write of the translated number by SD to fail.

When the translation of 29 to 781 does take place, the write to the Terminating_Number variable fails, specifically because of the block placed on it by the OCS feature. Since this is an inconsistency, it might appear that an interaction will be reported and the call aborted. However, the short-hand number that was dialled translates into a number which should have been permitted, so that is not the desired result.

What happens is that the constraint mechanism will backtrack and search for a solution along another ordering. For the example given, there is indeed such an order, which is shown in Fig 6. Note, that had the number dialled been 35, then it would have been guaranteed to have been translated into the number 666 first, before being subsequently rejected by OCS, as it should be.

3. Signal and resource contention

Several of the known instances of feature interaction are based upon conflicts dealing with call resources. Such a conflict can be based on two or more features attempting to manipulate a piece of hardware in different ways at the same time. It can also result from a feature's attempts to modify less concrete, yet equally important, call resources, like the roles which callers play within a call with respect to other callers. This sort of information, as indicated before, is maintained in the call sessions. Maintenance of call resource information in a centralised location with enforced uniform access facilitates the process of exposing interaction.

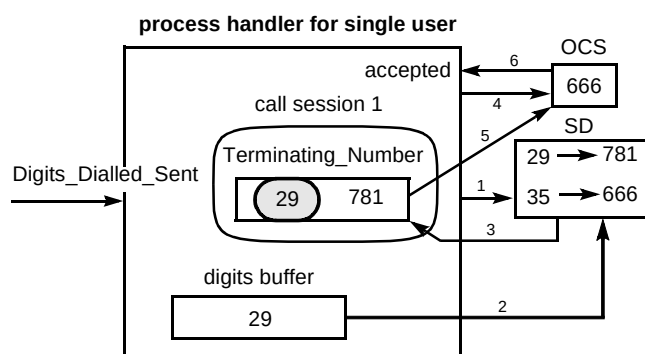


Fig 6 The number screening works correctly. SD translates the dialled digits 29 into the number 781 which OCS subsequently accepts. Note that if the digits dialled had been 35, then they would have been guaranteed to have been translated into the number 666 first before having been rejected by OCS.

3.1 Example

Consider the problem when the two features CW and three-way calling (3WC) are present together. When a user who has CW receives notification of an incoming call while on the line with another user, the switch-hook can be used to connect to the new caller, placing the initial caller on hold. Subsequent uses of the switch-hook toggles the two callers.

Three-way calling allows a busy user to include another caller in the present call. By activation of the switch-hook, the current call is put on hold and a dial tone is given with which to create a new call.

The interaction problem occurs when a call is placed to a busy caller, who has both CW and 3WC features. Arrival of a switch-hook signal is subject to ambiguous interpretation (see Fig 7).

3.2 Detection

In our proposed system, when the caller receives a connect request, CW will have been triggered and will have created a new call session with the caller as the terminating party. After the caller has been notified of the incoming call, if processed first by CW, then things will appear to be correct until processed by 3WC. At the point that 3WC attempts to start an outgoing session, the process will result in conflict, since CW will have already set and locked the type as a terminating session, rather than an originating one. Conversely, if 3WC processes the signal first, this violation is detected immediately, since the CW feature was already activated by the connection request signal, which will have preceded the switch-hook.

A very similar process occurs with the conflict between CFB and CW shown previously in Fig 1. Both of these features are triggered by an incoming connection request made to a busy caller. The feature allowed to process the signal first, whether CFB or CW, will create a new session and set the type of that session. When the other feature goes

to process the signal, a conflict is registered, since the types are incompatible and cannot be reset.

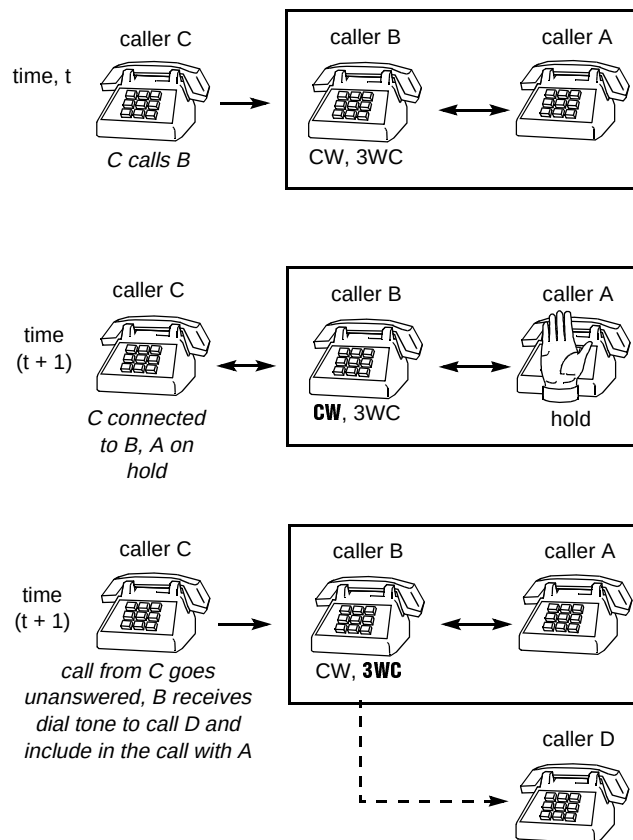


Fig 7 Interaction arises at time $(t + 1)$ — does the switch-hook signal cause CW to switch between callers or does 3WC give the caller a dial tone, permitting the dialling and inclusion of yet a different caller?

3.3 Resolution

In cases where interaction cannot be avoided, resolution is likely to involve the diminished or altered performance of at least some features. A constraint-based mechanism for resolution can be constructed with the constraints of Mittal [21], which allow for the conditional inclusion or exclusion of constraint subgraphs. At the point that it is determined that resolution is required, a constraint subgraph, like the one shown in Fig 8, can be included into the representation. It not only contains the constraints between each feature and the call resource, but also the constraints which allow a feature to defer its own functionality and permit the other feature to access the resource. This example was chosen merely to show that constraint-based resolution is possible. In cases where keeping the number of changes low is a concern, other constraint algorithms, such as the one for computing sets of minimal diagnoses [22], are appropriate and likely to be more useful.

4. Context violation

Most features place the caller in a certain role and take place in a particular circumstance. These conditions define a very specific context, and it is the violation of such

a context that can be attributed to many feature interactions. In particular, when a number of features are present, a caller can assume a wide variety of different roles with the features and other callers. Failure to keep track of such information can easily lead to confusion and ambiguity. Since it is generally outside the scope of the features to maintain this type of information, it becomes the responsibility of the feature interaction system to ensure the proper context.

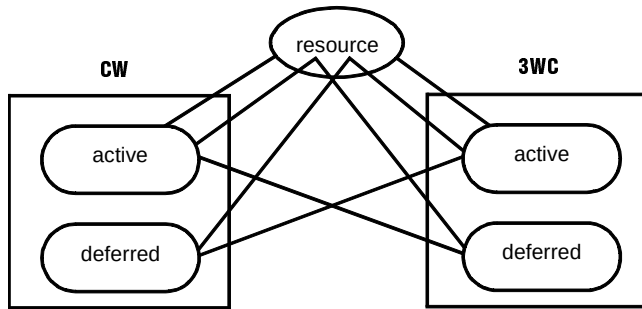


Fig 8 No solution with both CW and 3WC is possible, yet if one defers, there is a solution with the other.

4.1 Example

When CFB and OCS are used together, there is the potential for interaction. For example, consider the situation where *A* calls *B*, who is already engaged in a conversation. As the result of being busy, CFB is invoked, forwarding the call to another number *C*. Let us assume that *A* and *B* are free to call one another, i.e. neither appears on the other's screening lists, either OCS or TCS (see Fig 9). Now, there are several instances when interaction should be reported and some sort of explanation supplied.

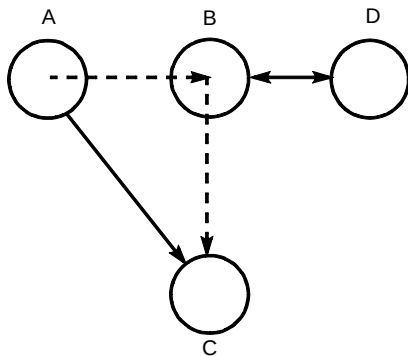


Fig 9 Caller *B* is busy with caller *D*, so the incoming call from *A* to *B* is forwarded by CFB to *C*.

- Assume *C* is on *A*'s originating call screening list, which means that calls from *A* to *C* are not permitted. When the call is forwarded, OCS will not permit the connection, and the caller will be notified that the calls to *C* are not permitted. However, caller *A* is likely to be confused, since it is not caller *C*, but rather caller *B*, that was called.

- Another, perhaps even more mystifying, problem would exist if *B* were on *C*'s terminating call screening list. When the call from *A* is forwarded to *C*, caller *A* might receive the message that caller *B* is not permitted to call. From the perspective of *A*, this has the potential to be completely confusing, since caller *A* is likely wondering 'What does caller *B* calling *C* have to do with my call?'

Given that call forwarding can be performed more than once, such scenarios have the possibility of becoming only more complicated and frustrating for callers, when connections are denied. The need for some sort of explanation becomes essential, especially in dealing with security-related issues, where service providers are likely to err on the side of caution, favouring aborting/denying even valid connections, rather than permitting even the occasional invalid ones.

4.2 Detection and resolution

Consider CFB from viewpoint of call recipient *B*, who receives a call request from *A*, but happens to be engaged in another call with *D* (see Fig 9). As the result of being busy, CFB is invoked and forwards the incoming call to another number *C*. The key to successful execution and interaction detection is the maintenance of the roles of each caller in each call session. The call recipient *B* plays two roles here, that of the terminating caller in *A* → *B* and the originating caller in *B* → *C* — so two separate sessions are generated, each defining its own context. These two sessions are in addition to the ones that *B* is involved with in the call it is already engaged in to *D* (see Fig 10).

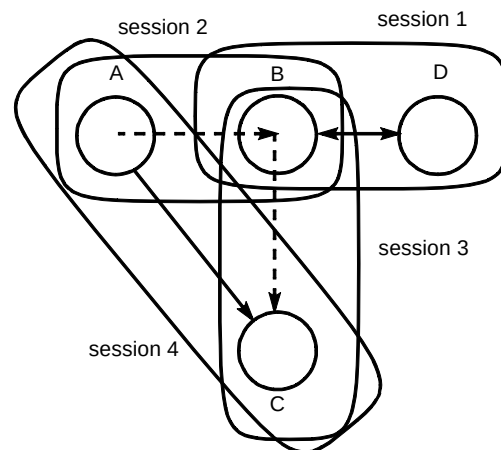


Fig 10 Session information maintained for the scenario in Fig 9. As caller *B* is busy with caller *D*, the incoming call from *A* to *B* is forwarded by CFB to *C*.

In this example, *B* and *D* are in one session, with *B* and *D* in their appropriate roles, *B* having called *D*. Now, when *A* calls *B*, *A* fills the originating, *B* the terminating slot of the session information. Yet, since *B* has CFB, a new call is initiated to the forwarded number *C*. What results is a session from *A* to *C* with *B* as the initiator. If it becomes

necessary to trace back to some other, more deeply nested participant, the session information of the other caller is also maintained. For example, if there were a question as to why CFB was initiated to forward A to C, the explanation containing session 1 would be extracted from the initiating field. It would reveal that B was engaged in a conversation with D.

Certainly, due to privacy and security issues, a caller might not be permitted to every detail of the cause of an interaction. However, such information could be invaluable to the service provider. Furthermore, resolution mechanisms might be able to reveal general characteristics of the source of the conflict without disclosing sensitive information. It is this hierarchically arranged maintenance of session information, reflecting how, when, and by whom actions were initiated, which exposes the roles of callers and their relationships with one another (see Fig 11).

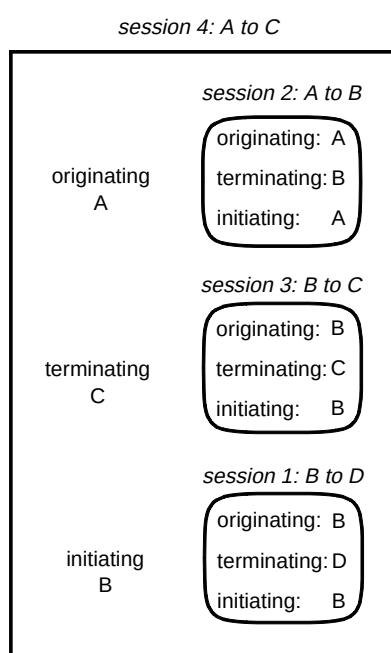


Fig 11 Session information maintained on the connection A to C, due to CFB forwarding the call to C.

Given that this detailed information is available for each session, the constraints of the OCS and TCS can be checked not only between the originating party and terminating parties, but also against the initiating party. As a result, the conflict in both scenarios (from the example section) can be explained. Once the interaction has been detected, this method has the advantage of identifying which callers are involved.

With this sort of context information, other interactions, like those that can occur with line credit features for B, can be easily exposed. Generally, calling parties with CFB are assessed the charge of the forwarded leg of the call, which in the example is the call from B to C. Very baffling interaction can occur if B has a feature invoked which

imposes some credit limit on calls. Exceeding the cost threshold can cause the connection between callers A and C to be severed, much to the surprise of either caller. At the very least, the call session information maintained can lead to an identification of the problem, even if the connection is broken. Ideally, however, the definition of the line credit feature would include some sort of notification before the connection is severed and provide the other calling parties an opportunity to assume the charges without dropping the call. Even if such an option is not explicitly provided, such functionality could be added to the resolution process, precisely because the context information reveals exactly what role B is playing in the call from A to C.

The call session information can also be easily extended to handle multi-party calls. To do so, additional fields are required for both active and inactive call participants. Augmented with these fields and accompanied by the appropriate constraints, problems with multi-party calls involving OCS, TCS, and looping are easily exposed.

5. Conclusions

The efforts to deal with run-time feature interaction [23] have led to the design of a system capable of avoidance, detection, and resolution under the umbrella of constraint technology. By equating consistency of the problem with interaction-free operation of the telecommunications system, interaction can be dealt with based on feature ordering, resource contention, and context

Two fundamental problems contributing to feature interaction have come to light from this work:

- the difficulty of determining and maintaining the state of call resources,
- the ambiguous interpretation of input signals due to context violations.

To combat these problems, ongoing efforts involve the development of a constraint-based framework of call resources within which features can be represented, and a constraint-based mechanism for the maintenance of context information to facilitate the understanding of the impact of signals.

Acknowledgments

This work is supported by British Telecommunications Laboratories. Special thanks to Steve Corley, Simon Tsang, and Richard Kett for assistance regarding telephony and intelligent networks, and to Nader Azarmi for general support.

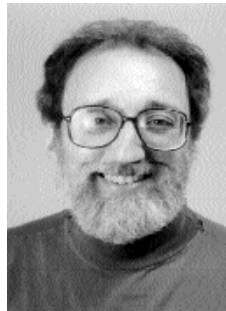
References

- 1 ITU-T Recommendation Q.1204: 'General recommendations on telephone switching and signalling; intelligent network: intelligent network distributed functional plane architecture', International Telecommunication Union (March 1993).
- 2 Cameron E J and Velthuijsen H: 'Feature interactions in telecommunications systems', *IEEE Communications Magazine*, **31**, No 3, pp 18—22 (August 1993).
- 3 Wakahara Y, Fujioka M, Kikuta H, Yagi H and Sakai S: 'A method for detecting service interactions', *IEEE Communications Magazine*, **31**, No 8, pp 31—37 (1993).
- 4 Cameron E J, Griffith N, Lin Y J, Nilson M E, Schnure W K and Velthuijsen H: 'A feature interaction benchmark for IN and beyond', in Bouma W and Velthuijsen H (Eds): 'Feature Interactions in Telecommunications Systems', IOS Press, pp 1—23 (1994).
- 5 Au P and Atlee J: 'Evaluation of a state-based model of feature interactions', in Dini P et al (Eds): 'Feature Interaction in Telecommunications Networks IV', IOS Press, pp 153—167 (1997).
- 6 Iraqi Y and Erradi M: 'An experiment for the processing of feature interactions within an object-oriented environment', in Dini P et al (Eds): 'Feature Interaction in Telecommunications Network IV', pp 298—312 (1997).
- 7 Khoumsi A: 'Detection and resolution of interaction between services of telephone networks', in Dini P et al (Eds): 'Feature Interaction in Telecommunications Networks IV', IOS Press pp 78—92 (1997).
- 8 Klein C, Prehofer C and Rumpe B: 'Feature specification and refinement with state transition diagrams', in Dini P et al (Eds): 'Feature Interaction in Telecommunications Networks IV', IOS Press, pp 284—297 (1997).
- 9 Nakamura M, Kakuda Y and Kikuno: 'Petri-net detection method for non-deterministic feature interactions and its experimental evaluation', in Dini P et al (Eds): 'Feature Interaction in Telecommunications Networks IV', pp 138—152 (1997).
- 10 Thistle J G, Malhame R P, Hoang H H, and Lafortune S: 'Feature interaction modelling, detection and resolution: a supervisory control approach', in Dini P et al (Eds): 'Feature Interaction in Telecommunications Networks IV', IOS Press, pp 93—107 (1997).
- 11 Frappier M, Mili A and Desharnais K: 'Detecting feature interactions on relational specification', in Dini P et al (Eds): 'Feature Interaction in Telecommunications Networks IV', IOS Press, pp 123—137 (1997).
- 12 Mierop J, Tax S and Janmaat R: 'Service interaction in an object-oriented environment', *IEEE Communications Magazine*, **31**, No 8, pp 46—51 (1993).
- 13 Bergstra B and Bouma W: 'Models for feature descriptions and interactions', in Dini P et al (Eds): 'Feature Interaction in Telecommunications Networks IV', IOS Press, pp 31—45 (1997).
- 14 Boumezeur R and Logrippo L: 'Specifying telephone systems in Lotos and the feature interaction problem', *IEEE Communications Magazine*, **31**, No 3, pp 38—45 (August 1993).
- 15 Capellmann C, Combes P, Ruiz J L, Petersson J and Renard B: 'Consistent interaction detection — a comprehensive approach integrated with service creation', in Dini P et al (Eds): 'Feature Interaction in Telecommunications Networks IV', IOS Press, pp 183—197 (1997).
- 16 Tsang S and Magill E H: 'Behaviour based run-time feature interaction detection and resolution approaches for intelligent networks', in Dini P et al (Eds): 'Feature Interaction in Telecommunications Networks IV', IOS Press, pp 254—270 (1997).
- 17 Griffith N D and Velthuijsen H: 'The negotiating agents approach to run-time feature interaction resolution', in Bouma W and Velthuijsen H (Eds): 'Feature Interactions in Telecommunications Systems', IOS Press, pp 217—234 (1994).
- 18 Utas G: 'Feature interaction: a software perspective', in Dini P et al (Eds): 'Feature Interaction in Telecommunications Networks IV', IOS Press, pp 23—28 (1997).
- 19 Sqalli M and Freuder E C: 'A constraint satisfaction model for testing emulated LANs in ATM networks', *Proceedings of the Seventh International Conference on Principles of Diagnosis*, pp 206—213 (1996).
- 20 Riese M: 'Diagnosis of extended finite state automata as a constraint satisfaction problem', *Proceedings of the Fourth International Conference on Principles of Diagnosis*, pp 60—73 (1993).
- 21 Mittal S and Falkenhainer B: 'Dynamic constraint satisfaction problems', *Proceedings of the Eighth National Conference on Artificial Intelligence*, pp 25—32 (1990).
- 22 Sabin D, Sabin M, Russell R D, and Freuder E C: 'Constraint-based approach to diagnosing software problems in computer networks', *Proceedings of the First International Conference on Principles and Practice of Constraint Programming (CP'95)* (1995).
- 23 Dini P, Boutaba R and Logrippo L: 'Feature Interaction in Telecommunications Networks IV', IOS Press (1997).



Charles Elfe holds a BS and an MS in computer science from the University of South Carolina in Columbia, SC, USA. He is an instructor and graduate student at the University of New Hampshire in Durham, NH, USA, where he is pursuing a PhD in computer science.

As a member of the Constraint Computation Center at UNH, he has been working to develop constraint-based techniques for feature interaction detection and avoidance in telecommunications. His current research interests also include network management and dynamic systems modelling and diagnosis.



Eugene C Freuder is a professor in the University of New Hampshire Department of Computer Science and Director of its Constraint Computation Center. He received his BA from Harvard and his PhD from MIT. He is a Fellow of the American Association for Artificial Intelligence. He is the founding editor of *CONSTRAINTS*, an international journal (Kluwer Academic Publishers), and executive chair of the Organizing Committee of the International Conference on Principles and Practice of Constraint Programming.



Society in 1997.

David Lesaint holds an MSc and a PhD in Artificial Intelligence from l'Ecole Nationale Supérieure de l'Aéronautique et de l'Espace (France). He has developed in his thesis a dual theory of Constraint Satisfaction problems for computing compact solution sets. He joined the Intelligent System Research group in January 1995 and since then has worked on the application of AI to various problems including Field Workforce Scheduling and Feature Interaction. He has designed and developed the planning algorithms of Dynamic Scheduler which was awarded an IT medal by the British Computer